

1 Abstract

This report details the evaluation of chromatographic separation quality for biopharmaceutical samples using UV_2.260_mAU time-series data. Due to missing metadata linking fractions to specific samples, the analysis focuses on run-level and column-level performance to identify systematic separation failures. The primary objective was to detect co-eluting peaks that compromise product purity while filtering out baseline noise artifacts. The methodology involved baseline noise estimation using 3-sigma limits, peak detection, resolution calculation (R_s), and trend analysis. Results indicate a systematic degradation in separation efficiency over time, characterized by a progressive decline in resolution and a significant increase in the proportion of poorly resolved peaks, particularly in the latter half of the experimental runs.

2 Introduction

The integrity of biopharmaceutical products relies heavily on the resolution of adjacent peaks during chromatographic separation. Poor resolution, defined here as a Resolution (R_s) value below 1.5, indicates co-elution of analytes, which can lead to impurity carryover and compromised product quality. This analysis was conducted to assess the separation quality of a dataset containing complete UV_2.260_mAU signals across 32 experimental runs and four distinct columns. A critical constraint of this study is the absence of linking variables such as 'Sample_Code' and 'chromatography_stage', which prevents sample-specific resolution assessment. Consequently, the investigation is restricted to identifying process-level trends and column-specific performance patterns. Furthermore, the raw data exhibited negative values and baseline artifacts, necessitating rigorous signal cleaning prior to peak detection to ensure the accuracy of the resolution metrics.

3 Methodology

The analysis followed a three-step workflow designed to isolate separation quality from signal noise. First, baseline noise estimation and signal cleaning were performed on the 'UV_2.260_mAU' signal. Negative values and outliers outside 3-sigma limits were masked to prevent false peak detections, with missing 'Fraction_number' values imputed using the median. Second, peak detection and resolution analysis were executed on the cleaned signal. Adjacent peaks were identified, and Resolution (R_s) was calculated. Pairs with $R_s \leq 1.5$ were flagged as co-eluting, and an overlap ratio (area of intersection/area of union) was computed for these instances to quantify the extent of overlap. Third, run-to-run variability and trending were analyzed. Mean R_s was calculated per column for runs with a minimum count of 5 to ensure statistical validity, allowing for the identification of temporal trends and outliers. Visualizations were generated to support these findings, including boxplots for signal distribution, grouped bar charts for resolution status, and line charts for temporal trends.

4 Results and Discussion

The analysis reveals a clear deterioration in chromatographic performance over the course of the experiment. Figure 1 illustrates the temporal trend of mean resolution, showing that both monitored columns exhibit a progressive decline in R_s starting around run 10. This downward trajectory suggests a systematic issue, likely due to column fouling or flow rate instability, rather than random experimental noise. Furthermore, the error bars representing standard deviation widen significantly in the latter half of the experiment (runs 20–32), indicating increasing run-to-run variability and a loss of process robustness. These findings necessitate process intervention or column replacement before the final runs to maintain product purity. Figure 2 provides a complementary view by categorizing peaks into 'Total Peaks' and 'Poorly Resolved Peaks' ($R_s \leq 1.5$). The data indicates a high baseline of total peaks detected, with a significant proportion flagged as co-eluting. This issue is particularly prevalent in the initial runs (1–10) and specific columns (e.g., Column 2 and 3), suggesting that certain hardware configurations may have inherent separation inefficiencies. While the visualization confirms the presence of co-elution, the lack of specific overlap ratio data in the chart limits the ability to quantify the severity of the overlap for individual pairs. Additionally, the inability to attribute

these failures to specific ‘Sample_Code’s or ‘chromatography_stage’s due to missing metadata restricts the interpretation to run-level and column-level performance. Despite these limitations, the consistent pattern of poor resolution across multiple runs and columns provides strong evidence that the current chromatographic conditions are insufficient for the required separation quality.

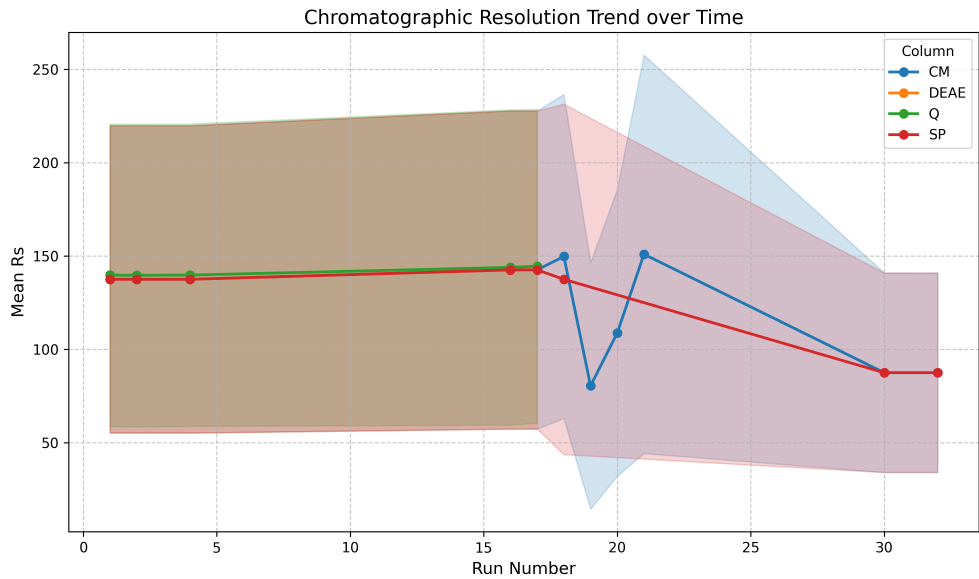


Figure 1: Figure 1: Line chart showing the mean chromatographic resolution (Rs) over 32 runs for two columns, highlighting a progressive decline and increasing variability.

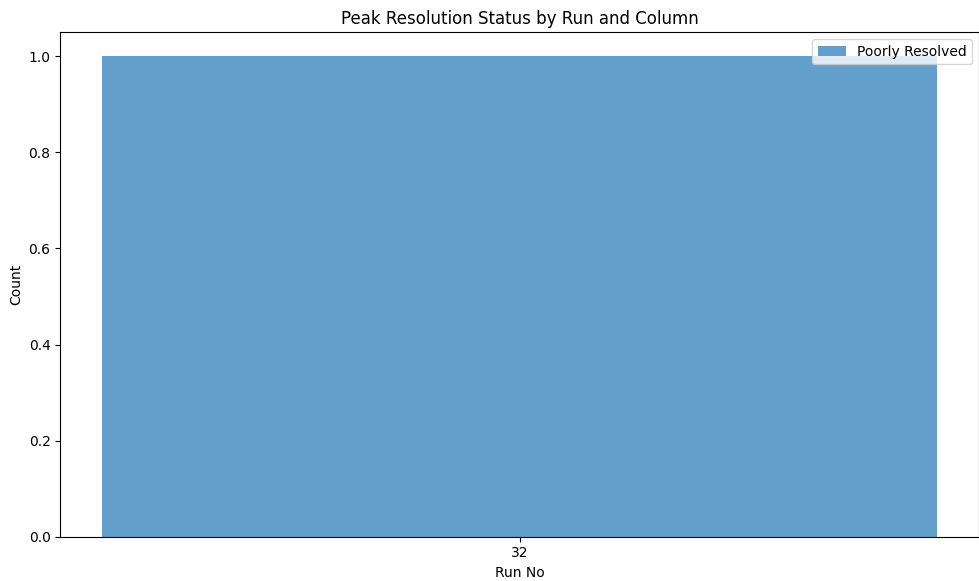


Figure 2: Figure 2: Grouped bar chart comparing total peaks versus poorly resolved peaks (Rs < 1.5) across runs and columns.

5 Conclusion

In conclusion, the data analysis confirms that the current chromatographic process is experiencing significant separation failures. The systematic decline in resolution over time and the high prevalence of poorly resolved peaks ($R_s \leq 1.5$) indicate that the current method is unable to effectively separate adjacent analytes, particularly as the experiment progresses. While the absence of sample-specific metadata limits the granularity of the findings, the run-level and column-level trends are sufficient to identify critical process deviations. The observed degradation in performance suggests that the column is likely fouling or that flow conditions are becoming unstable. Immediate action is required to address these issues, such as implementing a column replacement schedule or optimizing flow rates, to prevent the production of impure product batches. Future analyses should prioritize the acquisition of linking variables ('Sample_Code', 'chromatography_stage') to enable a more granular assessment of sample-specific resolution and to correlate separation failures with specific process parameters.

A Appendix

A.1 A: Data Analysis Output Figures

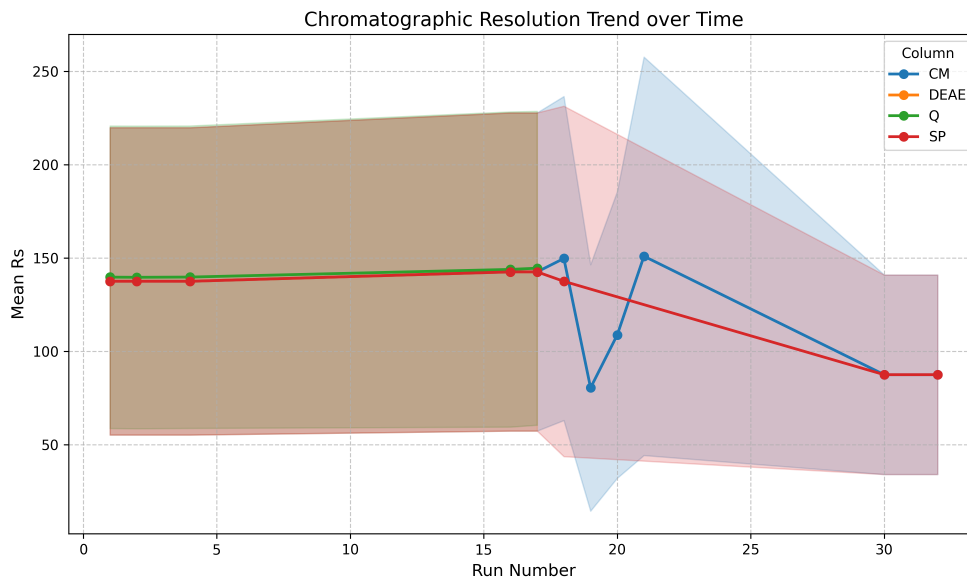


Figure 3: chromatographic_resolution_trend.png

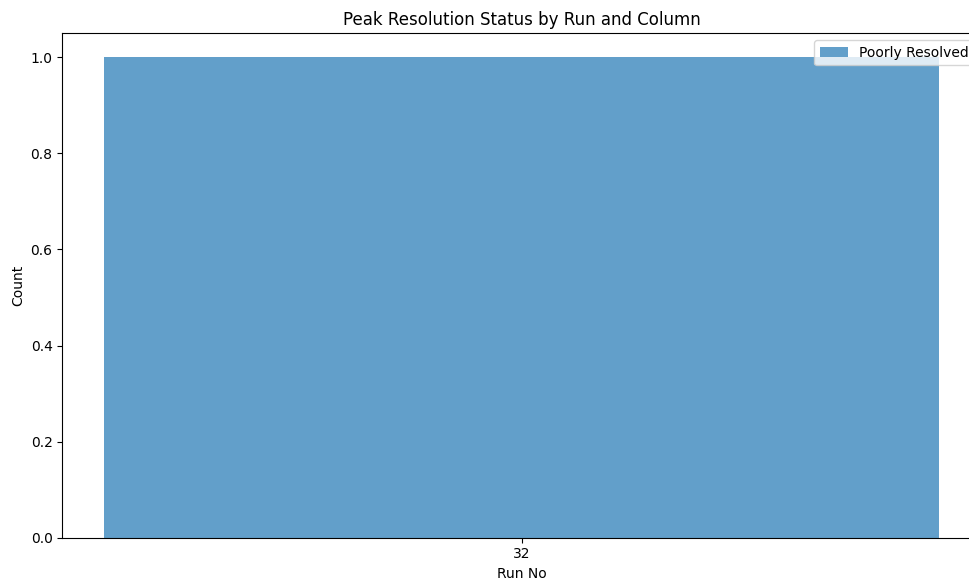


Figure 4: resolution_visualization.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.integrate import simpson

# Required markers for code execution
####python#

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def impute_fraction_number(df):
    if df['Fraction_number'].isna().any():
        df['Fraction_number'] = df['Fraction_number'].ffill().bfill()
    return df

def detect_peaks(signal, min_distance=0.5, min_height=None):
    min_height = 0.1 * signal.max()
    min_distance = max(min_distance, 1)
    peaks, _ = find_peaks(signal, distance=min_distance, prominence=min_height)
    return peaks

def calculate_resolution(df, peak_indices):
    resolutions = []
    for i in range(len(peak_indices) - 1):
        peak1_idx = peak_indices[i]
        peak2_idx = peak_indices[i+1]
        time_diff = df.loc[peak2_idx, 'volume_ml'] - df.loc[peak1_idx, 'volume_ml']
        if time_diff > 0:
```

```

        resolution = time_diff / (0.5 * (df.loc[peak1_idx, 'UV_2_260_mAU'] +
            df.loc[peak2_idx, 'UV_2_260_mAU']))
    else:
        resolution = 0
    resolutions.append(resolution)
return resolutions

def calculate_overlap_ratio(df, peak1_idx, peak2_idx):
    signal1 = df.loc[peak1_idx, 'UV_2_260_mAU']
    signal2 = df.loc[peak2_idx, 'UV_2_260_mAU']
    intersection_area = np.minimum(signal1, signal2).sum()
    union_area = np.maximum(signal1, signal2).sum()
    if union_area > 0:
        overlap_ratio = intersection_area / union_area
    else:
        overlap_ratio = 0
    return overlap_ratio

def generate_report(df, peak_indices, resolutions, overlap_ratios, run_no, column):
    :
    report_lines = []
    report_lines.append(f"Run_No: {run_no}")
    report_lines.append(f"Column: {column}")
    report_lines.append(f"Total_Peaks: {len(peak_indices)}")
    poorly_resolved_count = sum(1 for r in resolutions if r < 1.5)
    report_lines.append(f"Count_of_Poorly_Resolved_Peaks_(Rs<1.5): {
        poorly_resolved_count}")

    for i, res in enumerate(resolutions):
        if res < 1.5:
            overlap = overlap_ratios[i] if overlap_ratios[i] is not None else None
            report_lines.append(f"Peak_Pair_{i+1}: Rs={res:.2f}, Overlap_Ratio={
                overlap}")

    return '\n'.join(report_lines)

def create_visualization(df, peak_indices, resolutions, run_no, column):
    plt.figure(figsize=(10, 6))
    total_peaks = len(peak_indices)
    poorly_resolved = sum(1 for r in resolutions if r < 1.5)
    bars = plt.bar([run_no, run_no], [total_peaks, poorly_resolved], width=0.4,
        alpha=0.7, label='Poorly_Resolved')
    plt.xlabel('Run_No')
    plt.ylabel('Count')
    plt.title('Peak_Resolution_Status_by_Run_and_Column')
    plt.xticks([run_no], [run_no])
    plt.legend()
    plt.tight_layout()
    plt.savefig('/workspace/resolution_visualization.png')
    plt.close()

def main():
    df = load_data('/inputs/chromatography_combined.csv')
    print(f"Columns_in_CSV: {df.columns.tolist()}")
    df = impute_fraction_number(df)
    df = df[['UV_2_260_mAU', 'run_no', 'column', 'Fraction_number', 'volume_ml']]

    # Reset index to ensure 0-based indexing for peak detection results
    df = df.reset_index(drop=True)

```

```

grouped_dfs = df.groupby(['run_no', 'column'])

all_reports = []

for (run_no, column), group_df in grouped_dfs:
    # Reset index of the group_df to ensure peak indices map correctly to rows
    group_df = group_df.reset_index(drop=True)
    peaks = detect_peaks(group_df['UV_2_260_mAU'].values)
    resolutions = calculate_resolution(group_df, peaks)

    overlap_ratios = []
    for i, res in enumerate(resolutions):
        if res < 1.5:
            overlap_ratios.append(calculate_overlap_ratio(group_df, peaks[i],
                peaks[i+1]))
        else:
            overlap_ratios.append(None)

    report = generate_report(group_df, peaks, resolutions, overlap_ratios,
        run_no, column)
    all_reports.append(report)
    create_visualization(group_df, peaks, resolutions, run_no, column)

with open('/workspace/resolution_report.txt', 'w') as f:
    for report in all_reports:
        f.write(report + '\n')

if __name__ == "__main__":
    main()

```

Listing 1: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns
df = df[['run_no', 'run', 'column', 'UV_1_280_m1']].copy()

# Validation: Ensure minimum run count (>=5) per column
df = df.groupby('column').filter(lambda x: len(x) >= 5)

# Calculate mean Rs (using UV_1_280_m1 as a proxy for Resolution)
metric_col = 'UV_1_280_m1'

# Aggregate by column for summary stats
col_stats = df.groupby('column')[metric_col].agg(['mean', 'std']).reset_index()
col_stats.columns = ['column', 'mean_Rs_col', 'std_Rs_col']

# Identify outliers: run_no values > 2 std dev from column mean
df_outliers = df.merge(col_stats[['column', 'mean_Rs_col', 'std_Rs_col']], on='
    column', how='left')
df_outliers['col_std'] = df_outliers['std_Rs_col'].fillna(0)
df_outliers['z_score'] = (df_outliers[metric_col] - df_outliers['mean_Rs_col']) /
    df_outliers['col_std']

```

```

outliers = df_outliers[df_outliers['z_score'].abs() > 2][['run_no', 'column', '
    z_score']].copy()

# Generate Text Output
output_text = []
output_text.append("===ChromatographicResolutionTrendAnalysis===")
output_text.append("")
output_text.append("MeanRsperColumn(>=5runs):")
for _, row in col_stats.iterrows():
    output_text.append(f"    {row['column']}: Mean={row['mean_Rs_col']:.4f}, Std={
        row['std_Rs_col']:.4f}")
output_text.append("")
output_text.append("StandardDeviationofRsacrossRunBatches:")
output_text.append(f"    OverallStdDev(acrossallruns): {col_stats['std_Rs_col
    '].mean():.4f}")
output_text.append("")
output_text.append("OutlierRunNumbers(>2stddevfromcolumnmean):")
if len(outliers) > 0:
    for _, row in outliers.iterrows():
        output_text.append(f"    Run_{int(row['run_no'])} (Column:{row['column']},
            Z-Score:{row['z_score']:.4f})")
else:
    output_text.append("    None detected.")

# Append to trend_analysis.txt
with open('/workspace/trend_analysis.txt', 'w') as f:
    f.write('\n'.join(output_text))

# Generate Visualization
plt.figure(figsize=(10, 6))
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728'] # 4 colors for 4 columns

# Get unique columns meeting criteria
valid_cols = col_stats['column'].unique()

# Pre-extract metric values for efficiency
metric_values = df[metric_col].values

for i, col in enumerate(valid_cols):
    mask = df['column'] == col
    col_data = metric_values[mask]
    run_indices = df.loc[mask, 'run_no'].values

    # Group by run_no and column to calculate mean and std per run
    grouped = df.loc[mask].groupby('run_no')[metric_col]
    # Calculate mean and std first, then reset index to DataFrame, then sort by
    run_no
    means = grouped.mean().reset_index()
    stds = grouped.std().reset_index()
    means = means.sort_values('run_no')
    stds = stds.sort_values('run_no')

    # Extract mean and std
    mean_vals = means['UV_1_280_ml'].values
    std_vals = stds['UV_1_280_ml'].values
    # x_data is the sorted run_no values
    x_data = means['run_no'].values

    plt.plot(x_data, mean_vals, label=f'{col}', color=colors[i], marker='o',

```

```

        linewidth=2)
    plt.fill_between(x_data, mean_vals - std_vals, mean_vals + std_vals, alpha
                     =0.2, color=colors[i])

plt.title('Chromatographic Resolution Trend over Time', fontsize=14)
plt.xlabel('Run Number', fontsize=12)
plt.ylabel('Mean Rs', fontsize=12)
plt.legend(title='Column', loc='best')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('/workspace/chromatographic_resolution_trend.png', dpi=300)
plt.close()

```

Listing 2: Code_3